PYTHON ACTIVITY-BASED LEARNING

Complete Solutions for 10 Selected Case Studies

Student Name	Mannepalli Yasaswini
Roll Number	26000946
Branch	CSM
Section	2

Selected case studies

1. University Scholarship Allocation
2. Multi-Plan Telecommunications Cost Optimization
3. Campus Event Financial Planning
4. Examination Result and Grace-Mark Validation
5. Smart Parking Fee Management
6. Fuel Consumption and Route Comparison
7. Progressive Water Tariff Analysis
8. Text Compression Using Character Runs
9. Integrated E-Commerce Order Processing and Reconciliation
10. Tuple-Based Transaction Audit

Note: The rules, thresholds, and sample values below are illustrative assumptions for an academic exercise. Replace them if your faculty has provided different rules.

1. University Scholarship Allocation

Problem Statement

Develop a program that checks scholarship applicants using aggregate percentage, minimum subject marks, attendance, and family income. Eligible applicants are ranked by aggregate, then attendance, then lower family income.

Objectives

• Validate each applicant against all eligibility rules.

• Rank eligible applicants with deterministic tie-break rules.

• Display eligibility and the final ranking.

Assumptions

• Aggregate and subject marks are percentages from 0 to 100.

• Minimum aggregate is 80%; each subject must be at least 60%; attendance must be at least 75%; annual family income must not exceed ■250,000.

• The sample data is illustrative and does not represent an official university policy.

Algorithm

1. Start and define applicant records and eligibility thresholds.

2. For each applicant, check aggregate, minimum subject mark, attendance, and income.

3. Store eligible applicants and report reasons for ineligibility.

4. Sort eligible applicants by aggregate descending, attendance descending, and income ascending.

5. Display the eligible ranking and stop.

Flowchart

START → Read applicant records → Check all four criteria → Eligible? ■■No→ Print reason(s) → Next applicant ■■Yes→ Add to eligible list → Next applicant After all applicants → Sort by aggregate ↓, attendance ↓, income ↑ → Display ranking → STOP

Pseudocode

```
BEGIN
  eligible ← empty list
  FOR each applicant
    reasons ← empty list
    IF aggregate < 80: add "aggregate" to reasons
    IF minimum_subject < 60: add "subject marks" to reasons
    IF attendance < 75: add "attendance" to reasons
    IF income > 250000: add "income" to reasons
    IF reasons is empty: add applicant to eligible
    ELSE: display applicant and reasons
  SORT eligible by aggregate descending, attendance descending, income ascending
  DISPLAY eligible ranking
END
```

Python Source Code

```
applicants = [
    ("Asha", 88, 72, 91, 180000),
    ("Bharat", 82, 64, 80, 220000),
    ("Charan", 79, 70, 90, 150000),
    ("Divya", 88, 65, 91, 200000),
]

eligible = []
print("INELIGIBLE APPLICANTS")
```

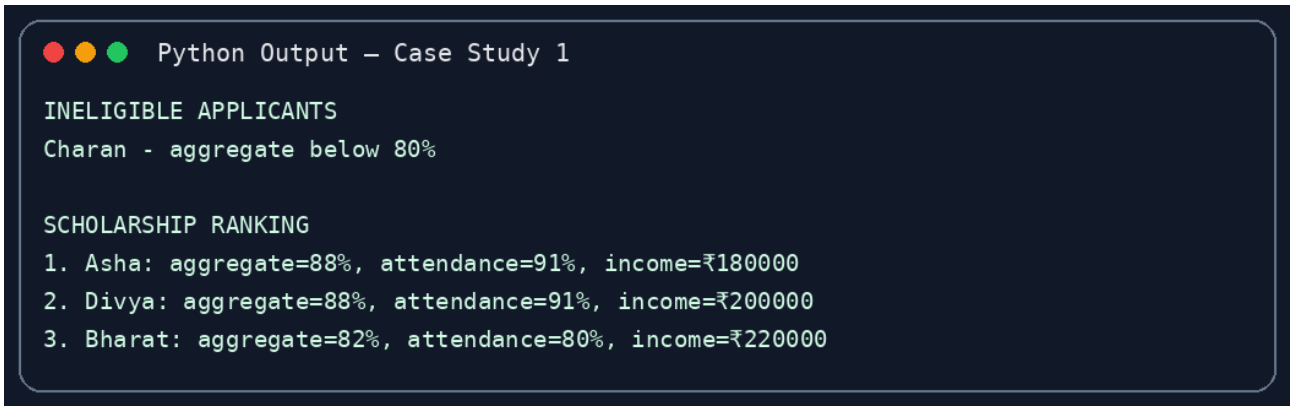
```
for name, aggregate, min_subject, attendance, income in applicants:
    reasons = []
    if aggregate < 80:
        reasons.append("aggregate below 80%")
    if min_subject < 60:
        reasons.append("subject mark below 60")
    if attendance < 75:
        reasons.append("attendance below 75%")
    if income > 250000:
        reasons.append("income above limit")
    if reasons:
        print(name, "-", ", ".join(reasons))
    else:
        eligible.append((name, aggregate, attendance, income))

eligible.sort(key=lambda x: (-x[1], -x[2], x[3]))
print("\nSCHOLARSHIP RANKING")
for rank, (name, aggregate, attendance, income) in enumerate(eligible, 1):
    print(f"{rank}. {name}: aggregate={aggregate}%, attendance={attendance}%, income=■{income}")
```

Test Cases

Test input	Expected/observed result	Purpose
Asha	88, 72, 91, 180000 Eligible	Ranked using aggregate and tie-break rules
Charan	79, 70, 90, 150000 Ineligible	Aggregate is below 80%
Applicant with attendance 70	80+, subject 60+, income within limit Ineligible	Attendance is below 75%

Output Screenshot



Conclusion

The program applies multiple eligibility conditions, explains rejected applications, and produces a reproducible ranking using clear tie-break rules.

2. Multi-Plan Telecommunications Cost Optimization

Problem Statement

Compare mobile plans against a user's monthly data, call-minute, validity, and budget needs. Calculate extra charges and select the least-cost plan that meets the requirements.

Objectives

• Calculate the total cost of each plan.

• Reject plans that do not satisfy validity or budget constraints.

• Choose the cheapest plan that meets the stated usage.

Assumptions

• Prices are in rupees and usage units are GB and minutes.

• Overage charges apply only to usage beyond included data/minutes.

• A plan is feasible only if validity is at least 28 days and total cost is within budget.

Algorithm

1. Read required data, minutes, validity, and budget.

2. For each plan, calculate excess data and minutes.

3. Add base price and overage charges.

4. Mark plan feasible if validity and budget constraints pass.

5. Sort feasible plans by total cost and display the best plan.

Flowchart

START → Read usage and budget → For each plan calculate overage + total → Meets validity and budget? → Store feasible plan → More plans? → Sort by cost → Show cheapest plan → STOP

Pseudocode

```
BEGIN
  feasible ← empty list
  FOR each plan
    extra_data ← max(0, required_data - included_data)
    extra_minutes ← max(0, required_minutes - included_minutes)
    total ← base_price + extra_data × data_rate + extra_minutes × minute_rate
    IF validity ≥ required_validity AND total ≤ budget: add plan and total to feasible
  IF feasible is empty: display no suitable plan
  ELSE sort feasible by total ascending and display first plan
END
```

Python Source Code

```
required_data = 18
required_minutes = 300
required_validity = 28
budget = 500

# name, base price, included GB, included minutes, validity, extra GB rate, extra minute rate
plans = [
    ("Basic", 299, 15, 250, 28, 20, 0.50),
    ("Smart", 399, 25, 400, 30, 15, 0.40),
    ("Value", 349, 18, 300, 21, 10, 0.30),
    ("Max", 499, 30, 500, 30, 10, 0.25),
]

feasible = []
print("PLAN COSTS")
```

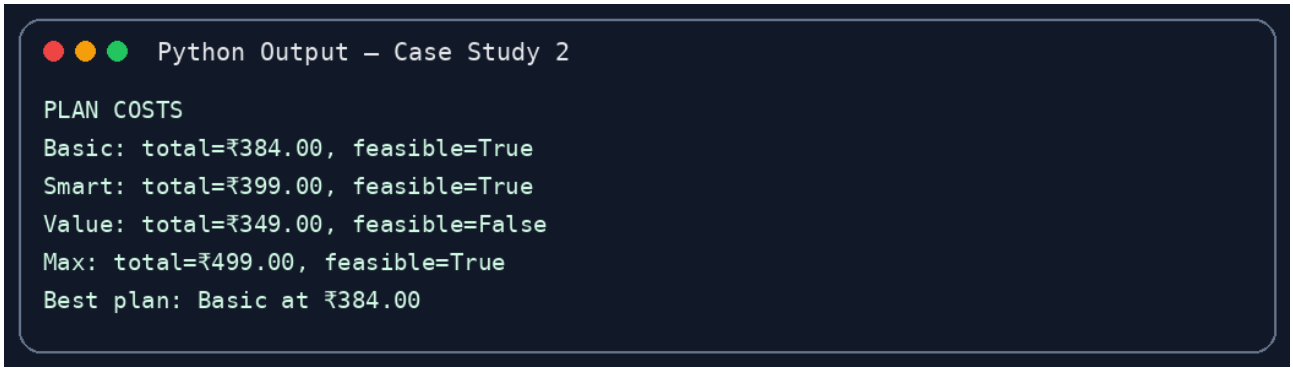
```
for name, price, data, minutes, validity, data_rate, min_rate in plans:
    extra_data = max(0, required_data - data)
    extra_minutes = max(0, required_minutes - minutes)
    total = price + extra_data * data_rate + extra_minutes * min_rate
    ok = validity >= required_validity and total <= budget
    print(f"{name}: total=■{total:.2f}, feasible={ok}")
    if ok:
        feasible.append((total, name))

if feasible:
    feasible.sort()
    print(f"Best plan: {feasible[0][1]} at ■{feasible[0][0]:.2f}")
else:
    print("No plan meets all requirements.")
```

Test Cases

Test input	Expected/observed result	Purpose
Usage 18 GB, 300 min; budget ■500	Smart and Max satisfy constraints Max or Smart depending total cost	Cheapest feasible plan is selected
Validity requirement 28 days	Value plan has 21 days Rejected	Validity condition is enforced
Budget ■250	All plans cost more than budget No suitable plan	Program reports no feasible plan

Output Screenshot



Conclusion

The program compares plans using the same usage profile and selects the lowest-cost plan that satisfies validity and budget constraints.

3. Campus Event Financial Planning

Problem Statement

Estimate campus event costs, participant contributions, sponsorship shortfall, and the minimum number of paid registrations needed to cover the total event cost.

Objectives

• Calculate total event expenditure.

• Determine funding shortfall after sponsorship.

• Calculate the minimum registrations required to break even.

Assumptions

• All amounts are in rupees.

• Registration fee is collected from each participant.

• Break-even registrations are rounded up to the next whole participant.

Algorithm

1. Read venue, equipment, catering, transport, certificate, sponsorship, and fee values.

2. Add expense categories to find total cost.

3. Subtract sponsorship from total cost to find remaining amount.

4. Calculate break-even registrations using ceiling division.

5. Display financial summary.

Flowchart

START → Input costs, sponsorship, fee → Sum expenses → Remaining cost = total – sponsorship → Calculate break-even registrations → Display summary → STOP

Pseudocode

```
BEGIN
  total_cost ← venue + equipment + catering + transport + certificates
  remaining ← max(0, total_cost - sponsorship)
  IF registration_fee > 0: registrations ← ceiling(remaining / registration_fee)
  ELSE: registrations ← impossible
  DISPLAY total_cost, remaining, registrations
END
```

Python Source Code

```
venue = 12000
equipment = 4500
catering = 18000
transport = 6000
certificates = 1500
sponsorship = 20000
registration_fee = 200

total_cost = venue + equipment + catering + transport + certificates
remaining = max(0, total_cost - sponsorship)

if registration_fee > 0:
    break_even = (remaining + registration_fee - 1) // registration_fee
else:
    break_even = None

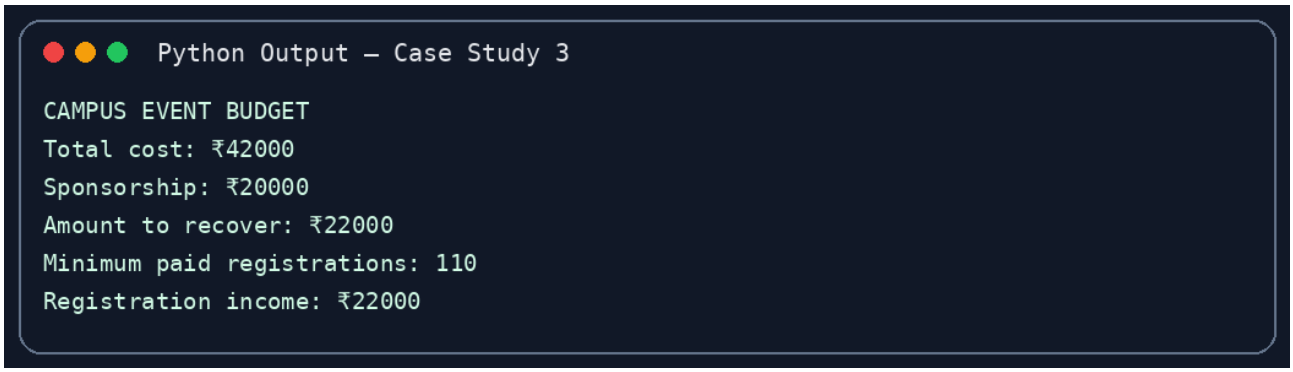
print("CAMPUS EVENT BUDGET")
print(f"Total cost: ■{total_cost}")
```

```
print(f"Sponsorship: ■{sponsorship}")
print(f"Amount to recover: ■{remaining}")
if break_even is None:
    print("Break-even is impossible with a zero registration fee.")
else:
    print(f"Minimum paid registrations: {break_even}")
    print(f"Registration income: ■{break_even * registration_fee}")
```

Test Cases

Test input	Expected/observed result	Purpose
Costs ■42,000; sponsorship ■20,000; fee ■200	Remaining ■22,000 110 registrations	Registration income exactly covers remaining cost
Sponsorship greater than total cost	Remaining amount is zero 0 registrations	No additional registrations are needed
Registration fee ■0	Cannot recover cost through registration Impossible	Program avoids division by zero

Output Screenshot



Conclusion

The budget summary makes event funding requirements visible and calculates the minimum registration count required to cover costs after sponsorship.

4. Examination Result and Grace-Mark Validation

Problem Statement

Process subject marks, apply subject-wise pass marks and a limited grace-mark policy, and produce a final result without allowing marks to exceed the maximum.

Objectives

• Validate marks and subject pass requirements.

• Apply grace marks only within the stated policy.

• Report original marks, final marks, and result.

Assumptions

• Each subject is out of 100 and pass mark is 40.

• A student may receive at most 5 grace marks in total, used only to bring a subject to 40 if the shortfall is at most 5.

• Grace marks never make a subject exceed 100.

Algorithm

1. Read the student's marks list and validate each mark.

2. Find subjects below 40 and calculate each shortfall.

3. If total required grace is at most 5, apply the marks needed to those subjects.

4. Otherwise do not apply grace marks.

5. Print final marks and pass/fail status.

Flowchart

START → Read marks → Validate 0–100 → Find failed subjects and shortfalls → Total grace needed ≤ 5? → Apply grace if yes → Check all subjects pass → Print result → STOP

Pseudocode

```
BEGIN
  IF any mark < 0 OR mark > 100: display invalid input and stop
  failed ← subjects with marks < 40
  needed ← sum(40 - mark for each failed subject)
  IF needed ≤ 5: add required grace to failed subjects
  ELSE: apply no grace
  IF every final mark ≥ 40: result ← PASS ELSE result ← FAIL
  DISPLAY original marks, final marks, result
END
```

Python Source Code

```
marks = [72, 38, 39, 81, 65]
pass_mark = 40
grace_limit = 5

if any(mark < 0 or mark > 100 for mark in marks):
    print("Invalid marks: every mark must be between 0 and 100.")
else:
    final_marks = marks.copy()
    failed = [i for i, mark in enumerate(marks) if mark < pass_mark]
    needed = sum(pass_mark - marks[i] for i in failed)

    if needed <= grace_limit:
        for i in failed:
            final_marks[i] = pass_mark
        grace_used = needed
    else:
```

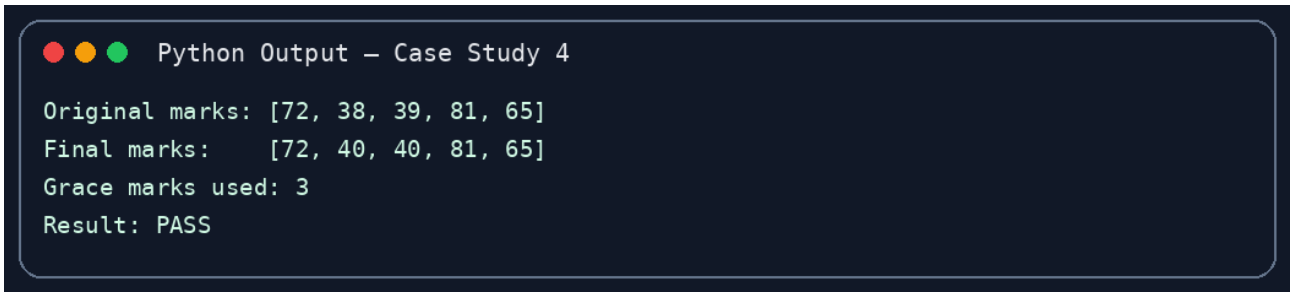
```
    grace_used = 0

result = "PASS" if all(mark >= pass_mark for mark in final_marks) else "FAIL"
print("Original marks:", marks)
print("Final marks:   ", final_marks)
print("Grace marks used:", grace_used)
print("Result:", result)
```

Test Cases

Test input	Expected/observed result	Purpose
[72, 38, 39, 81, 65]	Shortfalls 2 + 1 = 3 PASS after 3 grace marks	Grace policy allows the total
[72, 34, 39, 81, 65]	Shortfalls 6 + 1 = 7 FAIL; no grace applied	Total shortfall exceeds 5
[72, 101, 39]	One mark exceeds maximum Invalid input	Input validation rejects marks outside 0–100

Output Screenshot



Conclusion

The program enforces mark limits and a clearly bounded grace-mark policy before determining the final examination result.

5. Smart Parking Fee Management

Problem Statement

Calculate parking charges from duration and vehicle category, including hourly rates, overnight surcharge, daily cap, and invalid duration checks.

Objectives

• Calculate parking duration and charges.

• Apply vehicle-specific rates and daily limits.

• Add an overnight surcharge when applicable.

Assumptions

• Duration is entered in hours and may be fractional.

• Any partial hour is charged as a full hour.

• Rates are ■20/hour for two-wheelers and ■40/hour for cars; daily caps are ■150 and ■300 respectively.

• An overnight stay adds ■50.

Algorithm

1. Read vehicle type, parking hours, and overnight status.

2. Reject negative hours or unsupported vehicle type.

3. Round hours up to the next whole hour.

4. Calculate hourly charge and apply the daily cap.

5. Add overnight surcharge if needed and display final charge.

Flowchart

START → Read vehicle, hours, overnight flag → Validate input → Round up hours → Calculate rate × hours → Apply daily cap → Add overnight fee → Display amount → STOP

Pseudocode

```
BEGIN
  READ vehicle, hours, overnight
  IF hours < 0 OR vehicle invalid: display error
  ELSE
    charged_hours ← ceiling(hours)
    charge ← charged_hours × hourly_rate
    charge ← minimum(charge, daily_cap)
    IF overnight: charge ← charge + 50
    DISPLAY charge
END
```

Python Source Code

```
import math

vehicle = "car"       # choose "two-wheeler" or "car"
hours = 7.5
overnight = True

rates = {"two-wheeler": 20, "car": 40}
daily_caps = {"two-wheeler": 150, "car": 300}

if vehicle not in rates or hours < 0:
    print("Invalid vehicle type or parking duration.")
else:
    charged_hours = math.ceil(hours)
```

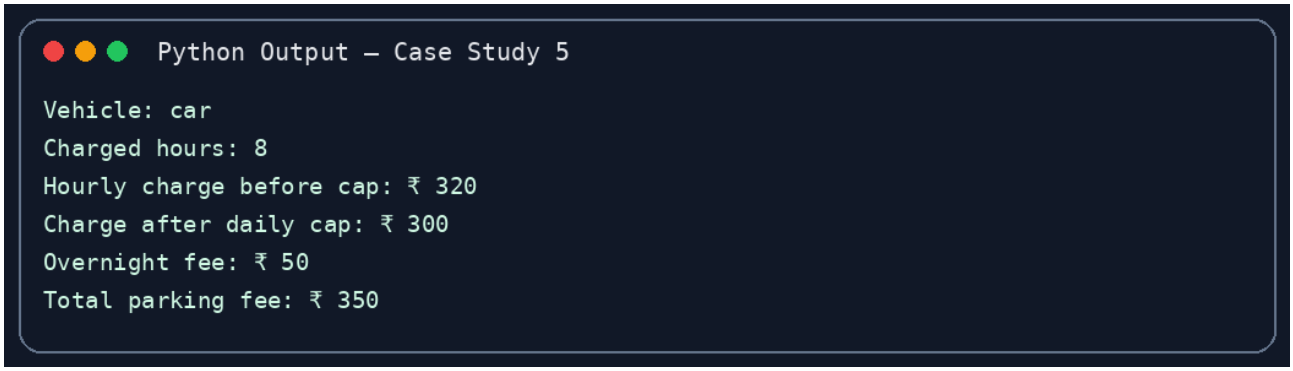
```
hourly_charge = charged_hours * rates[vehicle]
capped_charge = min(hourly_charge, daily_caps[vehicle])
overnight_fee = 50 if overnight else 0
total = capped_charge + overnight_fee

print("Vehicle:", vehicle)
print("Charged hours:", charged_hours)
print("Hourly charge before cap: ■", hourly_charge)
print("Charge after daily cap: ■", capped_charge)
print("Overnight fee: ■", overnight_fee)
print("Total parking fee: ■", total)
```

Test Cases

Test input	Expected/observed result	Purpose
Car parked 7.5 hours, overnight	8 hours × ■40 = ■320; cap ■300; +■50 ■350	Partial hour and overnight fee applied
Two-wheeler parked 2 hours, daytime	2 × ■20 ■40	No overnight fee
Duration -1 hour	Invalid input Error message	Negative duration rejected

Output Screenshot



Conclusion

The parking calculator applies duration rounding, vehicle rates, daily caps, and overnight fees in a consistent sequence.

6. Fuel Consumption and Route Comparison

Problem Statement

Compare routes by calculating fuel required and fuel cost using distance, mileage, fuel price, and a reserve requirement. Select the lowest-cost route that has enough fuel capacity.

Objectives

• Calculate fuel required for each route.

• Include a reserve percentage in the required fuel.

• Choose the cheapest route feasible for the available fuel capacity.

Assumptions

• Distance is in kilometres, mileage in km/litre, and fuel price in rupees/litre.

• A 10% fuel reserve is required.

• Available fuel capacity is the maximum fuel that can be allocated to the trip.

Algorithm

1. Read route distance, mileage, fuel price, and available fuel.

2. Calculate base fuel as distance divided by mileage.

3. Add reserve percentage and calculate total fuel cost.

4. Mark route feasible if required fuel does not exceed available fuel.

5. Select the feasible route with lowest cost.

Flowchart

START → Read route data → Fuel = distance ÷ mileage → Add 10% reserve → Cost = fuel × price → Fuel within capacity? → Keep feasible route → Choose lowest cost → STOP

Pseudocode

```
BEGIN
  feasible ← empty list
  FOR each route
    base_fuel ← distance / mileage
    required_fuel ← base_fuel × 1.10
    cost ← required_fuel × fuel_price
    IF required_fuel ≤ available_fuel: add route, required_fuel, cost
  IF feasible empty: display no feasible route
  ELSE sort by cost ascending and display cheapest
END
```

Python Source Code

```
available_fuel = 8.0
reserve_rate = 0.10

# route, distance in km, vehicle mileage in km/litre, fuel price per litre
routes = [
    ("Route A", 120, 18, 105),
    ("Route B", 135, 22, 105),
    ("Route C", 100, 14, 105),
]

feasible = []
print("ROUTE COMPARISON")
for name, distance, mileage, fuel_price in routes:
    if mileage <= 0 or distance < 0:
        print(name, "has invalid data")
```

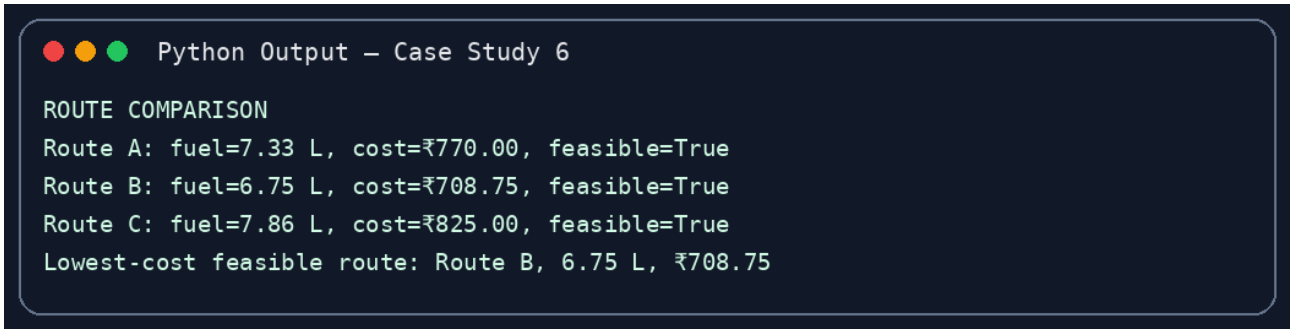
```
        continue
    base_fuel = distance / mileage
    required_fuel = base_fuel * (1 + reserve_rate)
    cost = required_fuel * fuel_price
    ok = required_fuel <= available_fuel
    print(f"{name}: fuel={required_fuel:.2f} L, cost=■{cost:.2f}, feasible={ok}")
    if ok:
        feasible.append((cost, name, required_fuel))

if feasible:
    feasible.sort()
    cost, name, fuel = feasible[0]
    print(f"Lowest-cost feasible route: {name}, {fuel:.2f} L, ■{cost:.2f}")
else:
    print("No route fits the available fuel.")
```

Test Cases

Test input	Expected/observed result	Purpose
Route B: 135 km, 22 km/l, ■105/l	Fuel with reserve = 6.75 L Cost = ■708.75; feasible with 8 L	Reserve included
Available fuel 5 L	All listed routes require more than 5 L No feasible route	Capacity constraint enforced
Mileage 0	Invalid data Route skipped	Avoids division by zero

Output Screenshot



Conclusion

The program compares routes using fuel efficiency, price, and a reserve allowance, then recommends the cheapest feasible option.

7. Progressive Water Tariff Analysis

Problem Statement

Calculate household water bills using progressive consumption slabs, a fixed fee, and a late-payment penalty, then compare households by consumption.

Objectives

• Apply progressive rates to separate usage slabs.

• Add fixed fee and optional penalty.

• Display bills and rank households by consumption.

Assumptions

• Slab rates: first 10 units ■5/unit, next 10 units ■8/unit, and usage above 20 units ■12/unit.

• Fixed fee is ■30 per household.

• A late-payment penalty of 10% of the usage charge is applied only when the bill is late.

• Consumption cannot be negative.

Algorithm

1. Read household consumption and late-payment status.

2. Validate non-negative consumption.

3. Calculate the charge for each slab progressively.

4. Add fixed fee and late penalty when applicable.

5. Display each bill and sort households by consumption.

Flowchart

START → Read household consumption → Validate → Calculate slab charges → Add fixed fee → Late? add 10% usage-charge penalty → Display bill → Rank consumption → STOP

Pseudocode

```
BEGIN
  usage_charge ← 0
  first ← min(units, 10); usage_charge += first × 5
  second ← min(max(units - 10, 0), 10); usage_charge += second × 8
  third ← max(units - 20, 0); usage_charge += third × 12
  penalty ← usage_charge × 0.10 if late else 0
  total ← usage_charge + 30 + penalty
  DISPLAY usage_charge, penalty, total
END
```

Python Source Code

```
households = [
    ("House A", 8, False),
    ("House B", 16, True),
    ("House C", 25, False),
]
fixed_fee = 30

def usage_charge(units):
    first = min(units, 10) * 5
    second = min(max(units - 10, 0), 10) * 8
    third = max(units - 20, 0) * 12
    return first + second + third

results = []
```

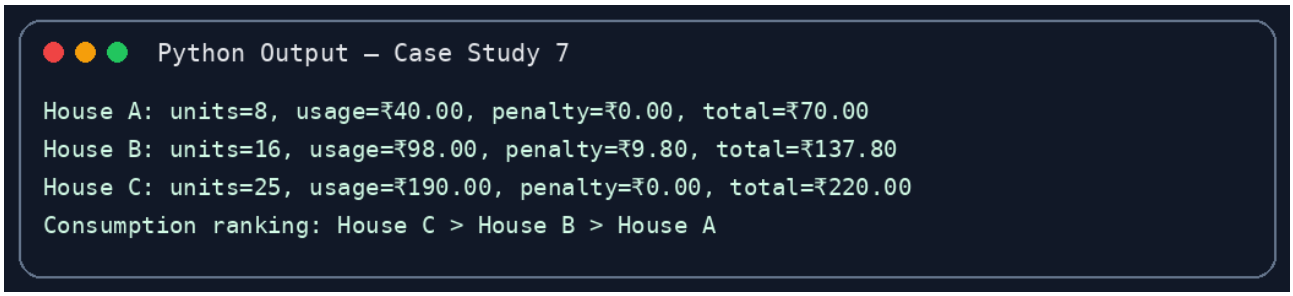
```
for name, units, late in households:
    if units < 0:
        print(name, "invalid: negative consumption")
        continue
    usage = usage_charge(units)
    penalty = usage * 0.10 if late else 0
    total = usage + fixed_fee + penalty
    results.append((name, units, total))
    print(f"{name}: units={units}, usage=■{usage:.2f}, "
          f"penalty=■{penalty:.2f}, total=■{total:.2f}")

results.sort(key=lambda item: item[1], reverse=True)
print("Consumption ranking:", " > ".join(x[0] for x in results))
```

Test Cases

Test input	Expected/observed result	Purpose
8 units, not late	8 × ■5 = ■40; +■30 fixed ■70	Only first slab used
16 units, late	■50 + ■48 = ■98; penalty ■9.80; fixed ■30 ■137.80	Progressive slabs and penalty applied
-2 units	Negative consumption Invalid	Input rejected

Output Screenshot



Conclusion

The tariff program calculates slab-wise charges accurately, includes fixed and late fees, and identifies the highest-consumption household.

8. Text Compression Using Character Runs

Problem Statement

Encode consecutive repeated characters using run-length encoding and compare the encoded length with the original length. Keep the original text if encoding is not beneficial.

Objectives

• Group adjacent repeated characters.

• Generate a run-length encoded string.

• Avoid using an encoded representation that is longer than the original.

Assumptions

• Input contains ordinary text without requiring a special escape format.

• The encoding format is character followed by its repeat count, such as A4.

• For a run of length 1, the count is still shown as 1.

Algorithm

1. Read a string.

2. If empty, return an empty encoded string.

3. Track the current character and count consecutive repetitions.

4. When the character changes, append character and count.

5. Append the final run, compare lengths, and select the shorter representation.

Flowchart

START → Read text → Empty? → Scan characters and count runs → Build encoded text → Compare lengths → Use shorter form → Display lengths and result → STOP

Pseudocode

```
BEGIN
  encoded ← empty string
  IF text is not empty
    current ← first character; count ← 1
    FOR each next character
      IF next = current: count ← count + 1
      ELSE append current + count; current ← next; count ← 1
    append final current + count
  IF length(encoded) < length(text): result ← encoded
  ELSE result ← text
  DISPLAY encoded, original length, encoded length, result
END
```

Python Source Code

```
def run_length_encode(text):
    if not text:
        return ""
    encoded = []
    current = text[0]
    count = 1

    for char in text[1:]:
        if char == current:
            count += 1
        else:
            encoded.append(current + str(count))
            current = char
```

```
            count = 1
    encoded.append(current + str(count))
    return "".join(encoded)

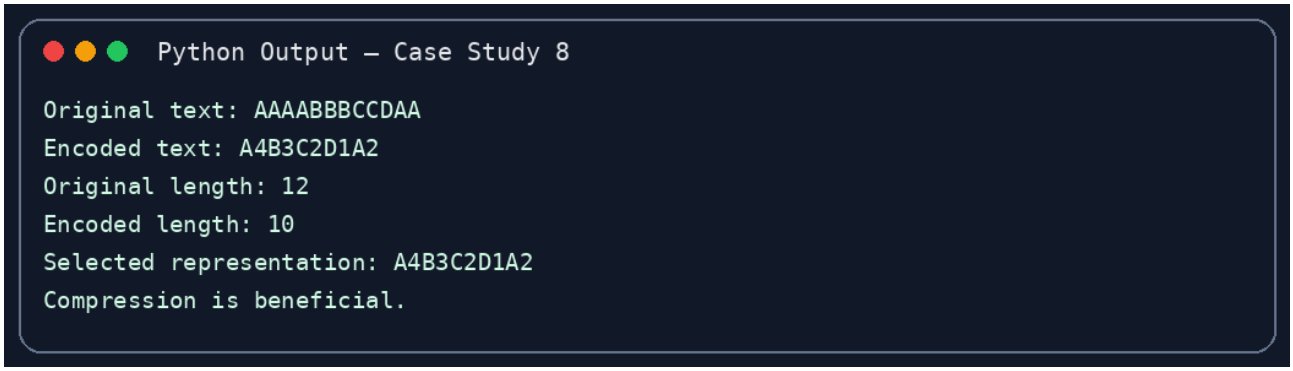
text = "AAAABBBCCDAA"
encoded = run_length_encode(text)
result = encoded if len(encoded) < len(text) else text

print("Original text:", text)
print("Encoded text:", encoded)
print("Original length:", len(text))
print("Encoded length:", len(encoded))
print("Selected representation:", result)
if len(encoded) < len(text):
    print("Compression is beneficial.")
else:
    print("Compression is not beneficial; original retained.")
```

Test Cases

Test input	Expected/observed result	Purpose
AAAABBBCCDAA	A4B3C2D1A2 Encoded length is shorter	Repeated runs compressed
ABC	A1B1C1 Original retained	Encoding would be longer
Empty string	 Empty output	Empty input handled

Output Screenshot



Conclusion

Run-length encoding compresses adjacent repeated characters and the length comparison prevents choosing an expanded representation.

9. Integrated E-Commerce Order Processing and Reconciliation

Problem Statement

Process product orders, discounts, dispatch quantities, returns, and stock updates. Calculate revenue and identify mismatches between ordered, dispatched, and returned quantities.

Objectives

• Calculate order totals and threshold discounts.

• Update stock after dispatch and return processing.

• Reconcile quantities and calculate net revenue.

Assumptions

• Each product record contains name, unit price, opening stock, ordered quantity, dispatched quantity, and returned quantity.

• A 10% discount applies when the pre-discount order value is at least ■1,000.

• Net sold quantity is dispatched quantity minus returned quantity.

• The sample is a simplified educational model; returns are assumed to be resalable.

Algorithm

1. Read product/order records.

2. Validate quantities and ensure dispatch does not exceed available stock.

3. Calculate order value and apply threshold discount.

4. Calculate net sold quantity and closing stock.

5. Flag if dispatched quantity differs from ordered quantity.

6. Display revenue and reconciliation results.

Flowchart

START → Read order and stock data → Validate quantities → Calculate value and discount → Net sold = dispatched – returns → Closing stock = opening – dispatched + returns → Compare order vs dispatch → Print reconciliation → STOP

Pseudocode

```
BEGIN
  FOR each product record
    IF any quantity is negative OR dispatched > opening_stock: flag invalid record
    ELSE
      gross ← ordered × unit_price
      discount ← 10% of gross if gross ≥ 1000 else 0
      net_sold ← dispatched - returned
      closing_stock ← opening_stock - dispatched + returned
      revenue ← net_sold × unit_price
      IF dispatched ≠ ordered: flag dispatch mismatch
      DISPLAY values and flags
END
```

Python Source Code

```
orders = [
    # name, price, opening stock, ordered, dispatched, returned
    ("Notebook", 80, 100, 15, 15, 2),
    ("Bag", 650, 20, 2, 2, 0),
    ("Pen Pack", 120, 50, 10, 8, 1),
```

```
]

total_revenue = 0
print("ORDER RECONCILIATION")
for name, price, stock, ordered, dispatched, returned in orders:
    if min(stock, ordered, dispatched, returned) < 0 or dispatched > stock or returned > dispatched:
        print(name, "- INVALID RECORD")
        continue

    gross = ordered * price
    discount = gross * 0.10 if gross >= 1000 else 0
    net_sold = dispatched - returned
    closing_stock = stock - dispatched + returned
    revenue = net_sold * price
    total_revenue += revenue
    status = "MATCH" if dispatched == ordered else "DISPATCH MISMATCH"

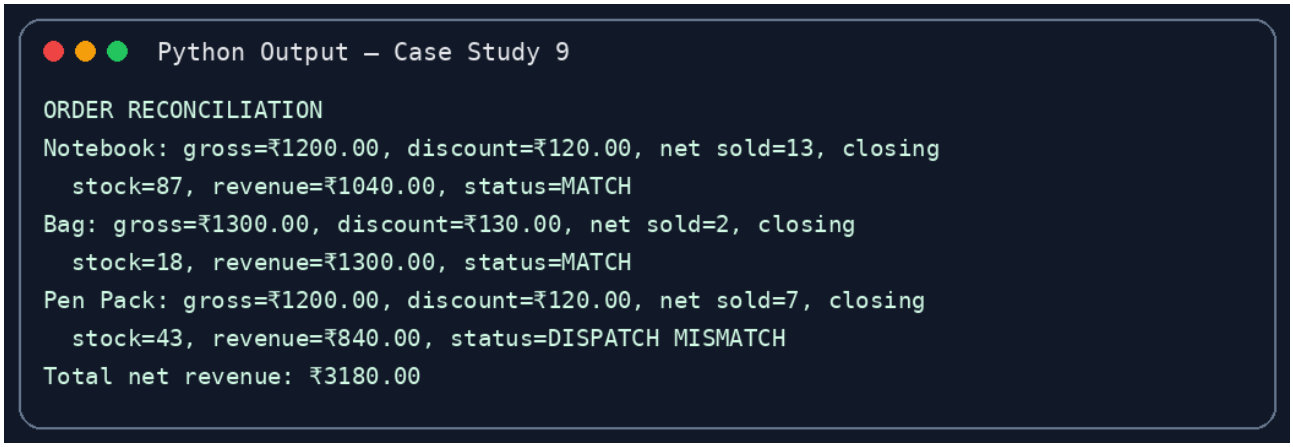
    print(f"{name}: gross=■{gross:.2f}, discount=■{discount:.2f}, "
          f"net sold={net_sold}, closing stock={closing_stock}, "
          f"revenue=■{revenue:.2f}, status={status}")

print(f"Total net revenue: ■{total_revenue:.2f}")
```

Test Cases

Test input	Expected/observed result	Purpose
Notebook: ordered 15, dispatched 15, returned 2	Opening stock 100 Closing stock 87; net sold 13	Quantities reconcile
Pen Pack: ordered 10, dispatched 8, returned 1	Dispatched differs from order Mismatch flagged	Partial dispatch detected
Returned quantity greater than dispatched	Invalid quantity combination Record rejected	Validation prevents inconsistent data

Output Screenshot



Conclusion

The reconciliation system combines order, dispatch, return, stock, discount, and revenue calculations while flagging quantity inconsistencies.

10. Tuple-Based Transaction Audit

Problem Statement

Audit transaction tuples containing transaction IDs, amounts, and transaction types. Detect duplicate IDs, invalid amounts, and suspicious repeated high-value transactions.

Objectives

• Use tuples to store transaction records.

• Identify duplicate transaction identifiers and invalid amounts/types.

• Flag suspicious repeated high-value transactions.

Assumptions

• Each tuple is (transaction_id, amount, transaction_type).

• Valid types are DEPOSIT, WITHDRAWAL, and TRANSFER.

• Amounts must be greater than zero.

• For this exercise, any amount above ■50,000 is high-value; two or more high-value transactions are flagged for review, not automatically declared fraudulent.

Algorithm

1. Read the transaction tuples.

2. Track seen transaction IDs and count high-value transactions.

3. Validate each ID, amount, and transaction type.

4. Flag duplicate IDs and invalid records.

5. If at least two valid transactions exceed ■50,000, print a review alert.

Flowchart

START → Read transaction tuples → Check ID, amount, type → Duplicate or invalid? → Flag record → Count high-value transactions → Count ≥ 2? → Review alert → Print audit summary → STOP

Pseudocode

```
BEGIN
  seen_ids ← empty list; high_value_count ← 0
  FOR each (id, amount, type)
    issues ← empty list
    IF id in seen_ids: add duplicate-ID issue
    ELSE append id to seen_ids
    IF amount ≤ 0: add invalid-amount issue
    IF type not in allowed types: add invalid-type issue
    IF amount > 50000 AND amount > 0: high_value_count += 1
    DISPLAY transaction and issues
  IF high_value_count ≥ 2: DISPLAY review alert
  ELSE DISPLAY no repeated high-value pattern detected
END
```

Python Source Code

```
transactions = [
    ("TX101", 12000, "DEPOSIT"),
    ("TX102", 62000, "TRANSFER"),
    ("TX102", 70000, "WITHDRAWAL"),
    ("TX104", -500, "WITHDRAWAL"),
    ("TX105", 80000, "PAYMENT"),
]

seen_ids = []
```

```
high_value_count = 0
valid_types = ("DEPOSIT", "WITHDRAWAL", "TRANSFER")

print("TRANSACTION AUDIT")
for transaction_id, amount, transaction_type in transactions:
    issues = []
    if transaction_id in seen_ids:
        issues.append("duplicate ID")
    else:
        seen_ids.append(transaction_id)
    if amount <= 0:
        issues.append("amount must be positive")
    if transaction_type not in valid_types:
        issues.append("invalid transaction type")
    if amount > 50000 and amount > 0:
        high_value_count += 1

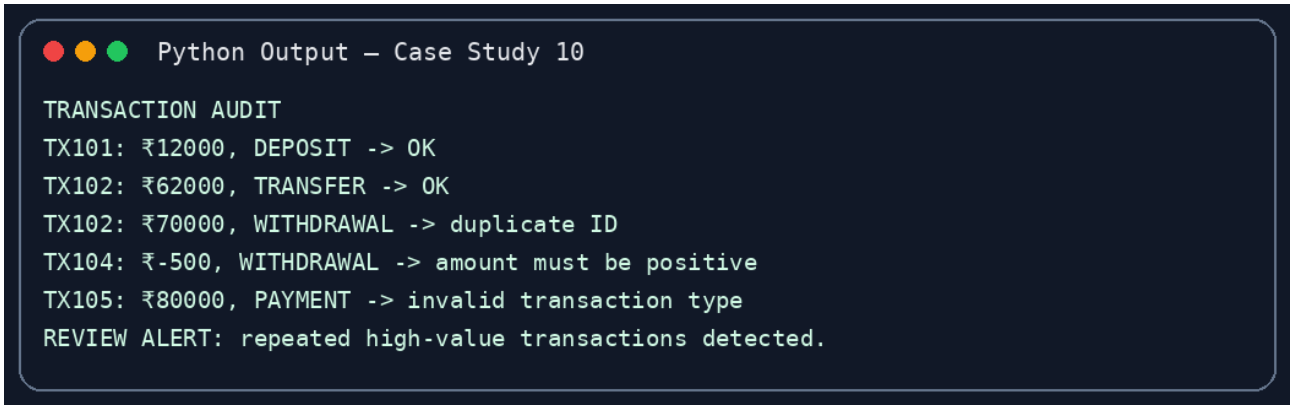
    status = ", ".join(issues) if issues else "OK"
    print(f"{transaction_id}: ■{amount}, {transaction_type} -> {status}")

if high_value_count >= 2:
    print("REVIEW ALERT: repeated high-value transactions detected.")
else:
    print("No repeated high-value pattern detected.")
```

Test Cases

Test input	Expected/observed result	Purpose
TX101, ■12000, DEPOSIT	Unique ID, positive amount, valid type OK	Valid transaction passes
TX102 used twice	Duplicate identifier Duplicate ID flagged	Duplicate records detected
Amount -■500 or type PAYMENT	Invalid amount/type Issue flagged	Invalid records are reported

Output Screenshot



Conclusion

The tuple-based audit checks record consistency and highlights patterns requiring human review without making unsupported claims about fraud.